

Smart Indoor Navigation System

Project Guide — Ek dum Simple Language Mein

A* Algorithm | Multi-Floor | Web App | College Campus

Review Draft — 2025

01. Introduction — What Are We Building?

Imagine you are new to a college campus. You don't know where the ICT Building is, or which floor has the HOD office. **GPS does not work indoors**. There is no app to help. You wander around, waste time, and feel lost. **This project solves exactly that.**

We are building a Web Application that:

- Shows a digital map of our college campus (starting with our own campus)
- Lets a user pick a Start point and an End point
- Instantly shows the SHORTEST path — including going up/down floors
- Clearly tells whether to use Stairs or Lift for floor changes

The brain behind shortest path calculation is a famous algorithm called **A* (A-Star)**. We will explain everything about it in simple words in this guide.

02. Problem Statement

Users inside large buildings face these real problems:

- Cannot find rooms, labs, or offices quickly
- Don't know which staircase leads to which floor
- No digital indoor map exists for most college campuses
- Time is wasted — especially in emergencies

Our Campus Scope (Phase 1)

Starting Point : Gate No. 9

Buildings : CME, CB, ICT (more to be added later)

Campus paths : Right fork -> Law College/School, Straight -> CME -> CB -> ICT

Multi-floor : Stairs and Lifts handled automatically

03. Core Concepts — Graph, A*, Heuristic

Before writing a single line of code, understand these **3 core concepts**. Everything in this project builds on these 3 ideas.

Concept 1 — Graph

A **Graph** is just a way to represent connections. Think of it like a metro map. Each station = a **Node**. Each track between stations = an **Edge**. The distance between stations = the **Weight**.

In Our Project

Node = Room / Gate / Junction / Stairs / Lift

Edge = Corridor or path between two nodes

Weight = Distance in meters (or pixels on the map image)

Concept 2 — A* Algorithm (A-Star)

A* is a step-by-step method that finds the **SHORTEST PATH** between two nodes in a graph. It is smarter than older algorithms like **Dijkstra** because it uses a **Heuristic** — a smart guess about how far the destination is.

The A* Formula (very important for viva):

```
f(n) = g(n) + h(n)

f(n) = Total estimated cost of this path (lower is better)
g(n) = Actual cost from START to current node (known)
h(n) = Heuristic: estimated cost from current node to END (guessed)

A* always picks the node with LOWEST f(n) next.
This way it avoids useless paths and reaches goal faster.
```

Why A* and NOT Dijkstra? (Viva Answer)

Dijkstra checks ALL nodes equally — slow.

A* uses h(n) to GUESS the direction toward the goal — skips useless nodes.

Result: A* is significantly faster. Simple as that.

Concept 3 — Heuristic Function h(n)

We use Euclidean Distance as our heuristic — the straight-line distance between two points:

```
Single Floor:
h(n) = sqrt( (x2-x1)^2 + (y2-y1)^2 )

Multi-Floor (add floor difference):
h(n) = sqrt( (x2-x1)^2 + (y2-y1)^2 + (floor2-floor1)^2 * floor_cost )

x, y = pixel coordinates of node on map image
floor_cost = extra penalty for changing floors
```

04. JSON — What It Is and How We Use It

JSON (JavaScript Object Notation) is just a way to write data in a text file so that any programming language can read it. Think of it as a very organized notebook. **Your entire campus map will live in ONE file: campus_map.json**

JSON's 3 Simple Rules

- Curly brackets { } = one object / item e.g. { "name": "Gate 9" }
- Square brackets [] = a list of items e.g. [item1, item2, item3]
- Everything is written as "key": value pairs. Numbers have no quotes.

Our Campus Map JSON — Full Example

The file has two sections: nodes (places) and edges (paths between places).

```
{
  "nodes": [
    { "id": "gate_9",    "label": "Gate No. 9",    "x": 100, "y": 300, "floor": 0,
      "type": "gate"    },
    { "id": "junc_1",    "label": "Junction 1",    "x": 200, "y": 300, "floor": 0,
      "type": "junction" },
    { "id": "law_col",    "label": "Law College",    "x": 350, "y": 150, "floor": 0,
      "type": "building" },
    { "id": "school",     "label": "School",         "x": 500, "y": 150, "floor": 0,
      "type": "building" },
    { "id": "cme_bldg",   "label": "CME Building",  "x": 350, "y": 300, "floor": 0,
      "type": "building" },
    { "id": "cb_bldg",    "label": "CB Building",   "x": 500, "y": 300, "floor": 0,
      "type": "building" },
    { "id": "ict_bldg",   "label": "ICT Building",  "x": 650, "y": 300, "floor": 0,
      "type": "building" }
  ],
  "edges": [
    { "from": "gate_9",  "to": "junc_1",  "weight": 10 },
    { "from": "junc_1",  "to": "law_col",  "weight": 25 },
    { "from": "law_col", "to": "school",   "weight": 20 },
    { "from": "junc_1",  "to": "cme_bldg", "weight": 30 },
    { "from": "cme_bldg", "to": "cb_bldg",  "weight": 20 },
    { "from": "cb_bldg", "to": "ict_bldg",  "weight": 20 }
  ]
}
```

What Each Field Means

Field	Meaning	Example
id	Unique name — for the computer	gate_9
label	Display name — shown to the user	Gate No. 9
x, y	Position on map image (in pixels)	100, 300
floor	Which floor (0 = Ground floor)	0, 1, 2
type	Category of this node	gate / stairs / room / junction
from/to	Which nodes this path connects	gate_9 to junc_1
weight	Distance — meters or pixels	10, 25, 30

Why Junction Nodes Are Important

At Gate 9, the path **splits** — right goes to Law College, straight goes to CME. That splitting point **MUST** be its own node called a **Junction**. Without it, A* cannot model the fork in the path correctly.

```
[Gate 9] --> [Junction 1] --> [CME] --> [CB] --> [ICT]
          |
          v
    [Law College] --> [School]
```

Every fork in the road = a Junction Node in your JSON.

05. Multi-Floor Navigation — The Secret

This is the part that makes our project impressive. The secret is simple: **Stairs and Lifts are just special nodes that connect two floors.**

Step-by-Step Example

User wants to go from ICT Entry (Floor 0) to HOD Office (Floor 2):

```
[ICT Entry - Floor 0]
|
v  walk to stairs
[ICT Stairs - Floor 0]
|
v  FLOOR CHANGE! (floor 0 -> floor 1)
[ICT Stairs - Floor 1]
|
v  FLOOR CHANGE! (floor 1 -> floor 2)
[ICT Stairs - Floor 2]
|
v  walk through corridor
[HOD Office - Floor 2]  REACHED!
```

Key Insight

A* does NOT know about floors. For A*, it is just nodes and edges.

Stairs Floor 0 is connected via an edge to Stairs Floor 1.

That edge has a slightly higher weight (climbing = more effort).

A* naturally picks this path if it is the shortest route.

JSON for Multi-Floor (ICT Building Example)

```
"nodes": [
  { "id": "ict_entry",    "label": "ICT Entry",    "floor": 0, "type":
"building" },
  { "id": "ict_stair_g",  "label": "ICT Stairs - GF",    "floor": 0, "type":
"stairs" },
  { "id": "ict_stair_f1", "label": "ICT Stairs - Fl. 1", "floor": 1, "type":
"stairs" },
  { "id": "ict_stair_f2", "label": "ICT Stairs - Fl. 2", "floor": 2, "type":
"stairs" },
  { "id": "lab_101",      "label": "Lab 101",      "floor": 1, "type":
"room" },
  { "id": "hod_office",   "label": "HOD Office",    "floor": 2, "type":
"room" }
],
"edges": [
  { "from": "ict_entry",    "to": "ict_stair_g",    "weight": 10 },
  { "from": "ict_stair_g",  "to": "ict_stair_f1",  "weight": 15 },    <- floor
change
  { "from": "ict_stair_f1", "to": "ict_stair_f2",  "weight": 15 },    <- floor
change
  { "from": "ict_stair_f1", "to": "lab_101",      "weight": 20 },
  { "from": "ict_stair_f2", "to": "hod_office",    "weight": 25 }
]
```

Frontend Floor Switch Logic

When A* returns the path, the frontend does this:

For each step in the path:

If `node[i].floor != node[i-1].floor`

-> Show alert: 'Take Stairs to Floor X'

-> Switch map image to that floor's image

-> Continue drawing path on the new floor map

06. Tech Stack and Project Structure

Layer	Technology	Why This?
Frontend (what user sees)	HTML + CSS + JavaScript	Simple, runs in browser, no install needed
Map Display	HTML Canvas or SVG	Draw map + path lines on top of it
Backend (brain)	Python + Flask	Simple web server, easy to learn
Algorithm	A* in Python	Fast, heuristic-based, great for viva
Data Storage	JSON file	No database needed, easy to edit by hand

Project Folder Structure

```
smart-nav/
|
+-- campus_map.json      <- All nodes + edges. Your entire map lives here.
|
+-- app.py               <- Flask backend + A* algorithm
|
+-- static/
|   +-- map_floor0.png   <- Ground floor map image
|   +-- map_floor1.png   <- First floor map image
|   +-- style.css
|
+-- templates/
    +-- index.html       <- Frontend UI (map display + dropdowns)
```

07. How the App Works — Full Flow

```
Step 1: User opens the web app in browser
Step 2: User selects SOURCE (e.g. Gate 9) from dropdown
Step 3: User selects DESTINATION (e.g. HOD Office) from dropdown
Step 4: User clicks 'Find Path' button
Step 5: Frontend sends request to Flask backend
        POST /find-path  body: { from: 'gate_9', to: 'hod_office' }
Step 6: Flask loads campus_map.json
Step 7: A* algorithm runs on the loaded graph
Step 8: A* returns path: ['gate_9', 'junc_1', 'cme_bldg', ... , 'hod_office']
Step 9: Flask sends path back to frontend as JSON
Step 10: Frontend draws a glowing line on map image using HTML Canvas
Step 11: If floor changes detected -> show alert + switch floor map image
```

What the User SEES on Screen

Map of campus with a coloured line showing the exact route
Step-by-step directions panel (like Google Maps turn-by-turn)
'Take Stairs to Floor 2' shown clearly when floor changes
Total distance displayed at the top

08. Implementation — Actual Code

A* Algorithm — Python

```
import heapq, math, json

def heuristic(n1, n2, nodes):
    dx = nodes[n1]['x'] - nodes[n2]['x']
    dy = nodes[n1]['y'] - nodes[n2]['y']
    df = (nodes[n1]['floor'] - nodes[n2]['floor']) * 50 # floor penalty
    return math.sqrt(dx*dx + dy*dy + df*df)

def astar(graph, nodes, start, end):
    open_set = [(0, start)] # (f_score, node_id)
    came_from = {}
    g_score = {start: 0}

    while open_set:
        _, current = heapq.heappop(open_set) # pick lowest f(n)

        if current == end: # GOAL REACHED
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1] # reversed = start to end

        for neighbor, weight in graph[current].items():
            new_g = g_score[current] + weight
            if new_g < g_score.get(neighbor, float('inf')):
                came_from[neighbor] = current
                g_score[neighbor] = new_g
                f = new_g + heuristic(neighbor, end, nodes)
                heapq.heappush(open_set, (f, neighbor))

    return [] # no path found
```

Flask Backend — app.py Skeleton

```
from flask import Flask, request, jsonify
import json

app = Flask(__name__)

# Load campus map once when server starts
```

```

with open('campus_map.json') as f:
    data = json.load(f)

@app.route('/find-path', methods=['POST'])
def find_path():
    body = request.json
    start = body['from']
    end = body['to']
    path = astar(graph, nodes, start, end) # call A* here
    return jsonify({ 'path': path })

if __name__ == '__main__':
    app.run(debug=True)

```

09. Step-by-Step Roadmap — What to Do First

Follow this order. Don't skip steps.

Phase	Task	Output
1 — Map	Physically walk campus. Sketch all nodes + junctions on paper.	Paper sketch
2 — Data	Assign x,y coords. Write campus_map.json file by hand.	campus_map.json ready
3 — Algorithm	Code A* in Python. Test with print statements first.	Working A* function
4 — Backend	Build Flask API with /find-path endpoint.	Running Flask server
5 — Frontend	Build HTML UI. Draw map + path line using HTML Canvas.	Visual map with path
6 — Multi-floor	Add floor nodes, stair edges, floor-switch logic in frontend.	Full multi-floor nav
7 — Polish	Dropdown menus, total distance, accessibility (lift preference)	Final web app

Start This Week — Your Actual Homework

1. Physically walk: Gate 9 -> CME -> CB -> ICT. Note every fork.
2. Sketch on paper. Every turn/fork = a junction node.
3. Take a photo of your sketch.
4. Open any text editor. Start writing campus_map.json.
5. That JSON file is your ENTIRE foundation. Everything builds on it.

10. Viva Q&A Cheat Sheet

Question	Answer
Why A* over Dijkstra?	A* uses heuristic $h(n)$ to estimate distance to goal. Dijkstra has no heuristic. A* skips useless nodes = faster.
What is your heuristic?	Euclidean distance between current node and destination using their x,y pixel coordinates on the map image.

How do you handle multi-floor?	Stairs/Lift nodes on each floor are connected by edges with higher weight. A* traverses these edges automatically.
Where is map data stored?	In a JSON file. Nodes store id, label, x, y, floor, type. Edges store from, to, weight.
What is a graph?	Data structure with Nodes (locations) and Edges (paths). Weighted edges represent distances between nodes.
What is $f(n) = g(n) + h(n)$?	$f(n)$ = total estimated path cost. $g(n)$ = actual cost from start. $h(n)$ = heuristic estimate to goal.
What if no path exists?	A* returns an empty list. Frontend shows 'No path found' message to user.